

related resources. If an application tries to use a service or resource for which it has no permission, then the service or resource is denied. The particular permission or capability is declared by the application developer, in the application's Manifest file. This level of security is over and above application sand-boxing. The important point here is that the Android security framework works evenly for Java code and native code, even though the native code is running outside the Dalvik VM.

Let's look at an example of how this model influences native code. When native code makes any calls to the Linux API 'socket', it needs to have the Android permission 'INTERNET'. Similarly, if native code tries to write to the SD card, it should have a permission known as 'WRITE_EXTERNAL_STORAGE', and so the list goes on.

The Android filesystem hierarchy

Traditional UNIX distributions follow a filesystem standard known as the Filesystem Hierarchy Standard (FHS). However, Android does not follow this or any other existing standard, as far as file system hierarchy is concerned. Various Android-specific file systems are mounted by the kernel at start-up, at not-very-standard mount points. So, native code should avoid making any assumptions about the availability of a specific mount point. Even if a particular mount point (or path like /dev, /tmp etc.) exists on the Android device, we will still not be sure if this is used in the same way it is used in Linux.

As mentioned earlier, Android provides application sand-boxing at the kernel level, which is implicitly applicable to the file system as well. For example, only that part of the file system is accessible to the application that has appropriate permissions, as determined by the application developer. However, there exists a concept of 'rooting' an Android device. This mechanism allows the user to escalate to 'root' user privileges. Once this is done, the user can access or modify anything on the file system. The reason for this is that the root user is not subject to any constraints or restrictions, even at the kernel level—in line with traditional Linux.

Here are some points to consider:

1. Although Android provides an application separation mechanism, the developer has an option to override this behaviour. You, as a developer, can decide whether your application needs to use this feature.
2. SysV IPC is not supported in the Android C run-time library because of security considerations; so native-code locking primitives that depend on SysV IPC, if any, need to be changed to use an alternative mechanism.
3. Android never stops a (Linux) process, unless the system is extremely low on resources. So, you should optimise the system to minimise memory consumption.
4. Android has tweaked the Linux kernel's timer mechanism (along with other features not directly

related here) in order to cater for the sleep/suspend/wake-up functionality of the device. If your native code uses any such mechanism, then make sure you check with the latest Android documentation.

5. Since any access to protected resources or services is guarded by the application permission framework, all access in native code needs to be analysed, and the required permissions should be identified. Whatever permissions the native code may need should be published for developers, so that they can include these permissions in their application's Manifest file.
 6. Native code should not rely on any code or mechanism that needs root access, as this won't be available on standard Android. Native code should rely only on the permissions governed by Android's application permission framework—which can be specified in the application Manifest file.
 7. Any hard-coded paths, like /tmp, which is in itself a bad practice, need to be removed, because such a path may not be available on the Android device, as discussed earlier. Yet, such code is still written, so you should scan your code to be sure!
 8. Android is designed for the smartphone and tablet market. These devices need to have a very reliable user-response mechanism. If an application does not respond within a specified time period, it's possible that the user is unable to switch to another application—which is annoying! In Android, this has been taken care of very well. If an application does not respond within a specified period of time (the default is 10 seconds—I'm not sure if this is configurable or not) then Android assumes that there is something wrong with the application, and prompts the user with a message like, "Application so-and-so not responding, do you want to stop it forcibly?". So, it's very important that native code does not wait for too long, like in locks, etc.
- As mentioned at the start, this is not a complete description of the Android security model. However, I believe this should give readers enough insight into the Android security framework for one to understand how it works, especially for native code developers. **END.** 

References

- [1] <http://developer.android.com/guide/index.html>
- [2] <http://developer.android.com/reference/android/Manifest.permission.html>

By: Raman Deep

The author is a principal software engineer with SafeNet's Software Rights Management division. He develops UNIX releases for products designed to help software vendors control how their applications are deployed and used. This article represents his views alone. He can be reached at rd.gdlinux@gmail.com.